



University of
Southern Denmark

Second Semester Project: Development of Software Systems (F26)

Your subtitle here

Group 11

Daniel Falat dafal25@student.sdu.dk

Dominik Bežo dobez25@student.sdu.dk

Maria Thalia Tsioni matsi25@student.sdu.dk

Pavel Seliavko pasel25@student.sdu.dk

Zoe Elea Schiwi zozei25@student.sdu.dk

Zoja Angelovicova zoang25@student.sdu.dk

May 25, 2026

Contents

1	Introduction	1
1.1	Problem Statement	1
1.1.1	Background	1
1.1.2	Motivation	4
1.2	Project Requirements and Objectives	5
1.2.1	Project Requirements	5
1.2.2	Project Objectives	6
2	Scrum Implementation	9
3	Sprints	11
3.1	Sprint 1	11
3.2	Sprint 2	12
3.3	Sprint 3	13
3.4	Sprint 4	15
3.5	Sprint 5	19
3.6	Sprint 6	19
4	Solution	21
4.1	UML Diagrams	21
4.1.1	Activity Diagrams:	21
4.1.2	Sequence Diagram:	22
4.1.3	Class Diagram:	23
4.2	Technical Architecture	24
5	Implementation and Technical Realization	27
5.1	Implementation	27
5.1.1	Implementation Frontend	27
5.1.2	Implementation Backend	29
5.1.3	Implementation Data Management	31
6	Discussion and Reflection	33
6.1	Scrum	33
6.2	Requirements Engineering	34

6.3	Refactoring	35
6.4	Code review	36
6.5	Testing	37
6.6	Future Work	39
7	Conclusion	41
A	Appendix A	43
A.1	First section	43
A.2	Second section	43
A.3	Third section	43

List of Tables

3.1	Start / Stop / Continue table for the 3rd Sprint Retrospective.	17
-----	---	----

List of Figures

1.1	District heating network of <i>Heatington</i>	2
1.2	Summer period graph	2
1.3	Winter period graph	3
1.4	Architecture of the system with relationships shown	7
3.1	Screenshot of the backlog before closing the 1st sprint.	12
3.2	Screenshot of the backlog after starting the 2nd sprint.	13
3.3	Screenshot of the backlog before closing the 2nd sprint.	13
3.4	Sprint 2 burnup chart. The drop in completed story points on 27 Mar reflects the Jira reporting issue described above, where points were re-attributed once parent tickets were closed.	14
3.5	Screenshot of the backlog after starting the 3rd sprint.	15
3.6	Screenshot of the backlog on the 16th of April.	16
3.7	Completed work items from Sprint 4.	18
3.8	Incompleted work items from Sprint 4.	18
3.9	Screenshot of the backlog after starting the 5th sprint.	20

Abstract

This is our abstract.

Chapter 1

Introduction

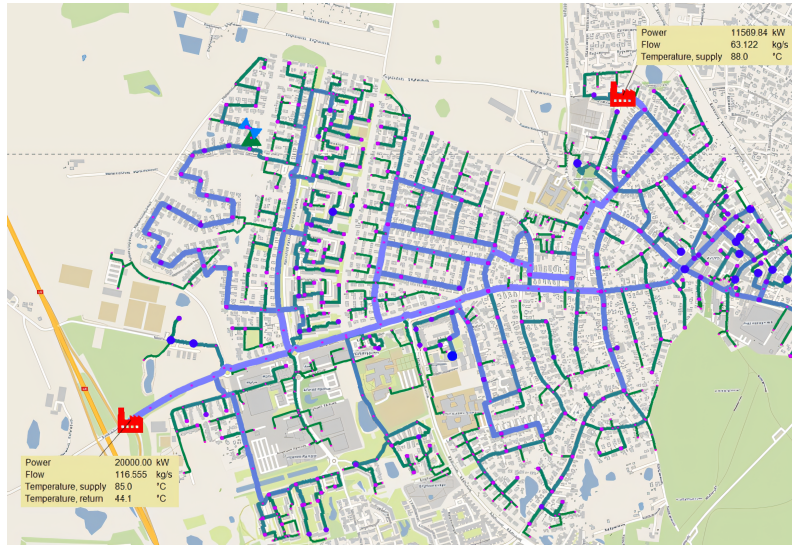
1.1 Problem Statement

The utility company *Danfoss* currently schedules their heat generation and distribution manually. Operators have to do all the calculations, compare every parameter for each production unit, track the electricity market prices hourly, plan the maintenance period, and schedule the heat generation and distribution all by themselves. As a result, the selection of unit can be non-optimal, the company misses on possible revenue from electricity production, they have greater expenses than needed, and there is a high risk and a great room for error that was influenced by the human factor. In addition, the manual system requires an overwhelming amount of time for the scheduling to be executed.

Danfoss seeks an automated software system that optimizes the heat generation and distribution hourly, aiming to obtain the greatest revenue possible, while minimizing the expenses. The proposed solution is a modular software system consisting of **Asset Manager**, **Source Data Manager**, **Result Data Manager**, **Optimizer**, and **Data Visualization**, developed in C# using the .NET framework and the MVVM architecture.

1.1.1 Background

Danfoss company is facing a significant challenge with scheduling the heat distribution to approximately 1600 buildings in the city *Heatington* through single district heating network. taking two different scenarios and two different periods into consideration when scheduling.

Figure 1.1: District heating network of *Heatington*

The system includes total of six production units, namely: **Gas Boiler 1**, **Gas Boiler 2**, **Gas Boiler 3**, **Oil Boiler 1**, **Gas Motor 1**, and **Electric Boiler 1**. Each operational scenario is defined by a different combination of the units available. In Scenario one, heat distribution is performed across one heating area by three gas boilers and one oil boiler. In Scenario two, heat distribution is performed across the same heating area, however, by using only two gas boilers, one gas motor, and one electric boiler. The system supports two seasonal operational periods, where both of these periods have specific values per each full hour for what the heat demand is, and what is the electricity cost at that time. The Summer period begins on 8th of September 2025 at 00:00 and ends on 21st of September at midnight, while the Winter period begins on 5th of January 2026 at 00:00 and ends on 18th of January at midnight.

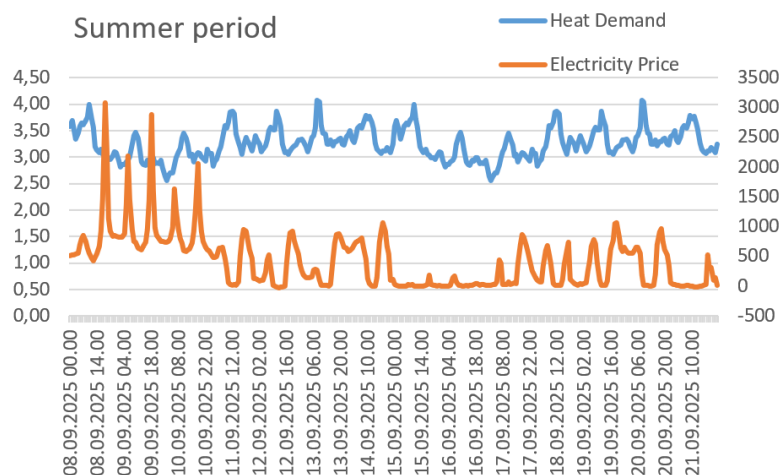


Figure 1.2: Summer period graph

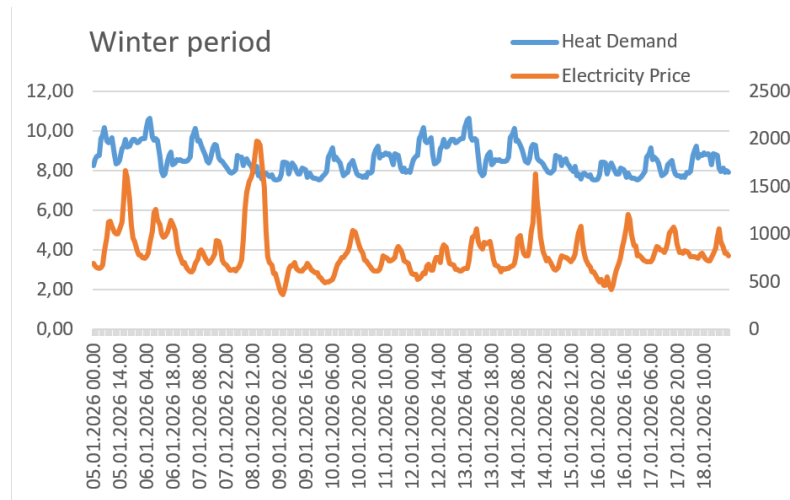


Figure 1.3: Winter period graph

The company is scheduling the distribution manually. This process requires operators to compare every aspect of all the production units, that depends also on the scenario and season, included the maximal heat the unit can produce, the cost of producing heat using this unit, the CO₂ emissions produced by the unit, the amount of fuel needed for generating the heat, and the maximal amount of electricity that could be produced. In addition, operators need to check all of the information for each period and do manual calculations to figure out what production unit should be used at a particular time, according to the objective of keeping the expenses as low as possible, while also seeking the highest possible gained revenue. One of the major challenges associated with executing the calculations is the cost of electricity, since its value is changing into a new, completely different value every hour, and it can also have a negative value. Another challenge is maintenance planning, which executed manually takes great amount of time when calculating and scheduling the heat distribution. Maintenance period must be scheduled, ranging from 30 to 60 hours, for one chosen production unit in each scenario once per year, preferably in the Summer period, since the heat demand is lower in this period.

As a result, the scheduling process takes an overwhelming amount of time. Furthermore, it also leads to greater expenses for the company than needed due to insufficient calculations. In addition, the company might miss on an opportunity to gain possible profit by not using the best production unit at the right time. Finally, the greatest flaw in this manual scheduling system is the possibility of the human factor being the root cause of these errors and challenges.

1.1.2 Motivation

Automizing and optimizing the scheduling of the heat distribution using the production units available results in reduction of the expenses needed, while also boosting the revenue gained from electricity produced. In addition, it will save the company's money for heat generation and distribution and increase the profit from electricity generation. Furthermore, it will make the process more efficient and it will eliminate the possibility of error made by a human factor.

Furthermore, the system can be designed in a way, where it could be reused for either another city, or another district heating area. In addition, it could also be reused for if there are new production units added, production units totally removed, or if the data and values for the existing production units parameters were to be changed. Ensuring scalability and flexibility for this software system would greatly improve its long-term usability.

One of the primary challenges in the optimization process is calculating the net production cost. For each production unit there is a predefined price for producing 1 MWh of heat. However, the cost for certain units that are using or producing electricity, the **Gas Motor 1** and **Electric Boiler 1**, can be different from the predefined price. The **Gas Motor 1** produces electricity as a byproduct that can be sold, which reduces its production cost. The **Electric Boiler 1** operates in an opposite way by consuming the electricity to operate, meaning the higher the electricity price is the more expensive it is to use the production unit. The **Optimizer** uses these net production costs to rank each unit from cheapest to the most expensive for each hour, and decides which unit should be used at a certain time.

In addition, there are also occurrences of the electricity prices having a negative value. This event flips the previously mentioned logic around, where the **Gas Motor 1** now creates expenses for producing the electricity, making it one of the most expensive units to run, and the **Electric Boiler 1** now gains revenue for consuming the electricity, making it potentially the cheapest, and better said, profitable option.

Furthermore, developing the **Data Visualization** module within the system will support the operators and the schedulers to view an easy to understand graph, turning complex optimization result data into intuitive and easily understandable format. The module displays the key information, namely: the heat demand, the heat production, the amount of electricity used, the electricity price, the CO₂ emissions produced, the fuel consumption, and the production cost. The graph will function in an intelligent way, where certain information that is special for particular scenario, for example the amount of electricity used and the electricity price, will only be available for viewing when the corresponding scenario is selected. In addition, the graph will provide an

interactive way to be operated, where the operator is able to click any part of the graph to view the information connected with the production unit that is used at the time. For instance, the date and time of the current selected point on graph, how much heat is generated by this unit, etc.

1.2 Project Requirements and Objectives

1.2.1 Project Requirements

Functional Requirements:

- **FR-1:** The system must secure heat for about 1600 buildings in the district heating area. It must guarantee that total heat production meets demand if a feasible solution exists.
- **FR-2:** The system must produce heat at the lowest costs. It must calculate "net production costs" for each unit per hour to find the cheapest way.
- **FR-3:** The system must use the electricity market to get the highest profit (from selling) or lowest costs (from buying). It should handle negative prices where you get paid to consume electricity.
- **FR-4:** The system has to consist of 5 modules, namely: **Asset Manager**, **Source Data Manager**, **Result Data Manager**, **Optimizer**, and **Data Visualization**.
- **FR-5:** The system must be able to operate in two distinct operational scenarios.
 - **FR-5.1:** Scenario 1 = The system must operate for a district heating area using 3 Gas Boilers (**Gas Boiler 1**, **Gas Boiler 2**, **Gas Boiler 3**) and one Oil Boiler (**Oil Boiler 1**).
 - **FR-5.2:** Scenario 2 = The system must operate for a district heating area using 2 Gas Boilers (**Gas Boiler 1**, **Gas Boiler 3**), one Gas Motor (**Gas Motor 1**), and one Electric Boiler (**Electric Boiler 1**).
- **FR-6:** The system must be able to operate in two distinct, 14-days long seasonal periods, in Summer and in Winter.
- **FR-7:** The system must allow for a maintenance period to be scheduled between 30 and 60 hours where one unit is offline and not used by the **Optimizer**.

Non-functional Requirements:

- **NFR-1 (Modularity):** The system must have a modular architecture so components can be replaced or extended easily.

- **NFR-2 (Time Resolution):** All data and calculations must use a strict 1-hour time resolution.
- **NFR-3 (Tech Stack):** Developed in .NET using an object-oriented language like C#.
- **NFR-4 (Storage):** The system must use local file formats like csv or excel for data.
- **NFR-5 (Usability):** The GUI should let users configure scenarios and run optimization without editing config files directly.
- **NFR-6 (Testing):** The code should follow good practices and include tests for the subsystems.

1.2.2 Project Objectives

The objective of this project is to develop a software system for the utility company *Danfoss*, which is responsible for heat generation and distribution to approximately 1600 buildings in the city *Heatington*, through a single district heating system. The proposed solution is that the system must be capable of operating in two distinct scenarios where there are different production units available. Furthermore, the system must be able to operate throughout two different 14 days long periods, in summer and in winter. The automatization and optimized scheduling of production units helps with delivering sufficient amount of heat according to the heat demand at the lowest possible cost. Furthermore, it maximizes the possible profit to be gained from the electricity generation.

The primary objectives of this project are:

- Develop a modular software system consisting of 5 different modules, namely:
 1. **Asset Manager (AM)** - acts as a source repository for static system information, responsible for retrieving the data when called by the **Optimizer** module or the **Data Visualization** module
 2. **Source Data Manager (SDM)** - acts as a source repository for dynamic system information, responsible for retrieving the data when called by the **Optimizer** module or the **Data Visualization** module
 3. **Result Data Manager (RDM)** - acts as a sink repository for optimization results, responsible for saving the optimized data from the **Optimizer** module into CSV files for each unit, and retrieving that data when called by the **Data Visualization** module
 4. **Optimizer (OPT)** - serves as a kernel module for optimization of the energy production

5. **Data Visualization (DV)** - responsible for visualizing the optimized result data in a clear and understandable format

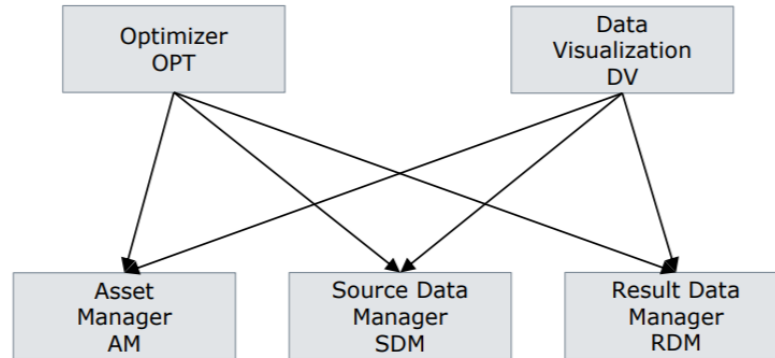


Figure 1.4: Architecture of the system with relationships shown

- Develop the logic for the **Optimizer** module, so that it can generate optimization of the heat production and distribution schedule with the lowest possible operational cost, and with the highest possible revenue
- Design the system to support two operational scenarios and two different seasonal periods, the summer period and the winter period
- Design the **Result Data Manager** so it stores all of the optimized result data into CSV files with the name identical to the production unit name
- Develop a GUI using the programming language C# and the Avalonia module and its library
- Implement a control panel in the GUI for the operational scenarios switching
- Implement a maintenance control panel in the GUI for scheduling the production unit maintenance period
- Implement the functionality that allows the user to switch between dark and light theme in order to improve the user experience

Chapter 2

Scrum Implementation

Chapter 3

Sprints

This chapter documents the work carried out during each of the project's six sprints. For every sprint the goal, the team composition, the major events of the Daily Standups, the outcome of the Sprint Review and the conclusions drawn during the Sprint Retrospective are reported.

3.1 Sprint 1

The first sprint focused on establishing the ground of the project. The sprint goal was "Create design tools for all of the modules and establish the project infrastructure", which consisted of the Noun-Verb analysis, CRC cards and UML diagrams for each of the 5 modules (Asset Manager, Source Data Manager, Result Data Manager, Optimizer and the Data Visualizer). Alongside was established the project infrastructure work: linking Jira with GitHub, creating the initial Avalonia project, converting the XLSX file with data into a CSV file and storing it in the assets folder.

Due to differences in experience and skillset levels inside the group, the team was split into three pairs:

- Maria Tsioni and Zoja Angelovicova
- Pavel Seliavko and Zoe Elea Schiwy
- Dominik Bezo and Daniel Falat

Dominik Bezo acted as the Scrum Master, Pavel Seliavko acted as the Meeting Coordinator and after discussion with the supervisor, Radoslaw Marek Chylak, the role of the Product Owner was assigned to Radoslaw Marek Chylak.

During the first Daily Standup it became clear there was space for adding more tasks to the sprint. After consulting with the supervisor a Figma prototype was added to the Sprint Backlog.

Throughout the Sprint Review, all of the design tools tickets had the Noun–Verb analysis approved and marked as done, several CRC cards and UML diagrams required further improvements, therefore these items were moved back to the Product Backlog.



Figure 3.1: Screenshot of the backlog before closing the 1st sprint.

The main conflicts were discussed during the Sprint Retrospective:

- Dismissiveness and resistance to different ideas
- Inactivity, mainly during online meetings
- Lack of immediate feedback
- Underestimation of the workload

These issues were resolved by trying to adopt a more receptive approach, agreeing on turning on cameras during online meetings, raising issues immediately rather than waiting for the Sprint Retrospective and adding a custom story point system (Difficulty 1–5 + Importance 1–5) to better collaborate and estimate workload in future sprints.

3.2 Sprint 2

With a clearer picture of the workflow, and after improving certain design tools, Sprint 2 focused on the actual implementation of the first two modules: the Asset Manager and the Source Data Manager. Building on the Figma prototype completed in Sprint 1, the first version of the GUI was also built during this sprint. Each task was further divided into smaller subtasks.

Pavel Seliavko obtained the role of the Scrum Master while also keeping the role of the Meeting Coordinator. The original three pairs from Sprint 1 remained unchanged

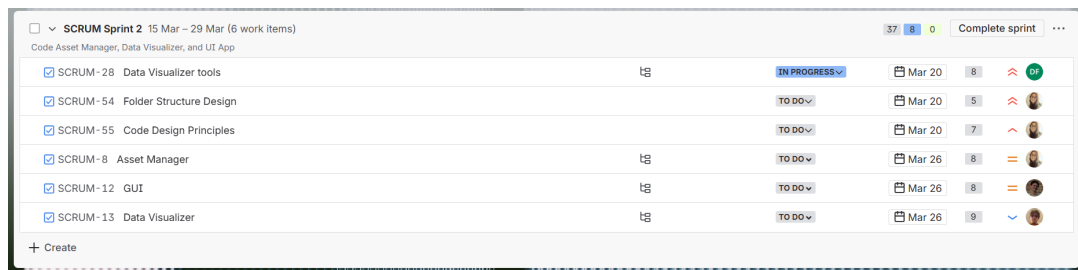


Figure 3.2: Screenshot of the backlog after starting the 2nd sprint.

and the agreed story point system was applied to each ticket by each team (e.g. the Source Data Manager was rated 5 for importance and 4 for difficulty).

Early in the sprint, during the first Daily Standup, it was recognised that the initial plan – which focused on the Data Visualizer – was not realistic, therefore the task was replaced with the implementation of the Source Data Manager. This exchange showed that the initial planning did not account for the full complexity of the whole implementation.

The Sprint Retrospective concluded that one of the pairs assigned for pair-programming struggled to find common working hours due to a difference in time schedules, which slowed down the progression of tickets assigned to this team. This issue was resolved by rotation of the pair-programming partners in the following sprint, purposefully pairing up teammates with more aligned schedules.

Sprint 2 successfully delivered the Asset Manager, the Source Data Manager and the first version of the GUI. The initial burndown chart appeared incorrect because Jira did not register the completed subtasks until the whole tickets were moved to the “DONE” category. This issue was realised at the very end of the sprint.

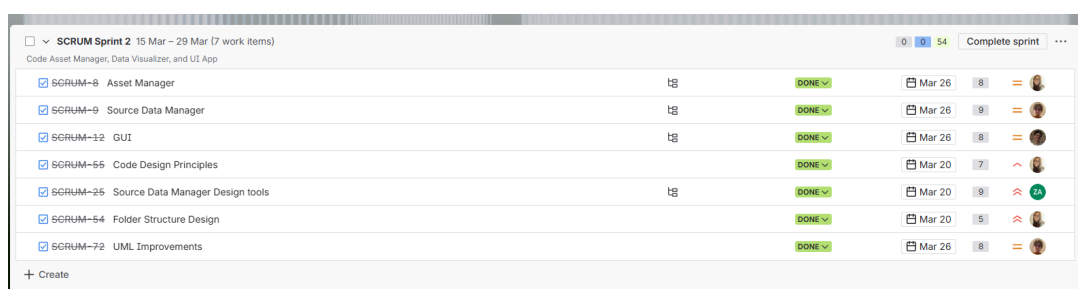


Figure 3.3: Screenshot of the backlog before closing the 2nd sprint.

3.3 Sprint 3

Sprint 3 focused on the remaining modules: the Optimizer, the Result Data Manager and the Data Visualizer. Zoja Angelovicova acted as the Scrum Master and Dominik Bezo as the Meeting Coordinator, while Pavel Seliavko transitioned into the role

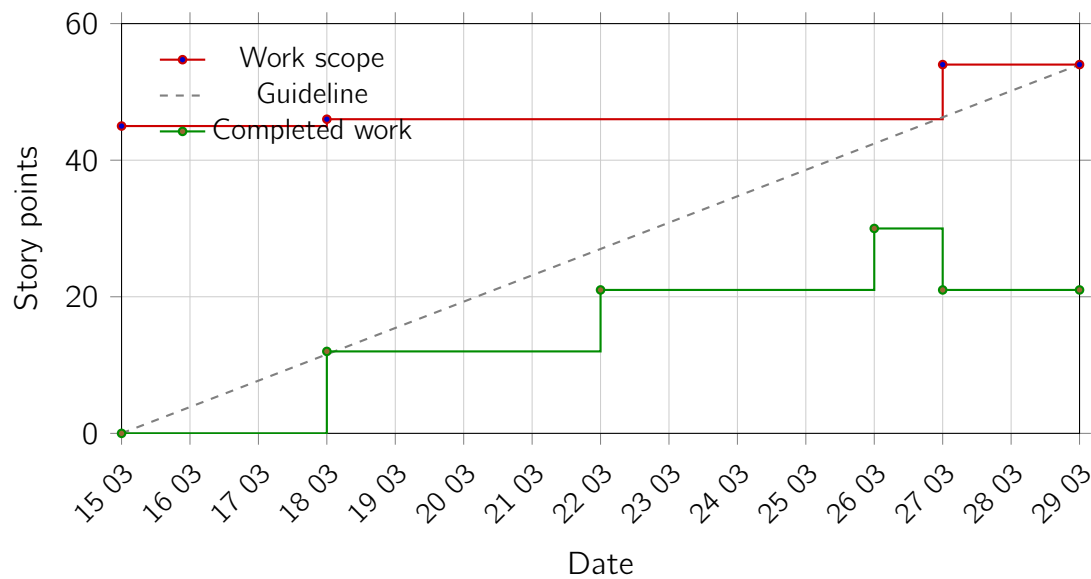


Figure 3.4: Sprint 2 burnup chart. The drop in completed story points on 27 Mar reflects the Jira reporting issue described above, where points were re-attributed once parent tickets were closed.

of internal Product Owner. To address the mid-sprint creation and replacement of tickets, a problem identified in Sprint 2, the planning style was changed. Each pair created its own detailed subtickets for the assigned module before starting development. Because of the issue with the burndown chart from Sprint 2, every subtask was created as a separate ticket so that Jira would track progress correctly.

The pairs were also rearranged based on schedule compatibility:

- Maria Tsioni and Dominik Bezo
- Zoja Angelovicova and Daniel Falat
- Pavel Seliavko and Zoe Elea Schiwy

Sprint 3 delivered a working Optimizer, the Result Data Manager and a Data Visualizer, but during the merging of parallel branches integration errors occurred and forced the sprint to be extended until the 19th of April instead of the 16th of April as initially planned. A few of the tickets remained outside the DONE column at the official end of the sprint because of this. The conflicts were resolved by a full team merge. All of the branches were merged into a single branch `SCRUM-81-Version-1`. Furthermore several tickets had to be renamed to reflect what had actually been implemented.

The Retrospective used a Start / Stop / Continue board with silent writing and round-robin sharing methods. The outcome is summarised in Table 3.1.

SCRUM Sprint 3 6 Apr – 23 Apr (37 work items) Code the Optimizer, Result Data Manager and Data Visualizer 227 8 0 Complete sprint ...			
☒ SCRUM - 13 Data Visualizer		TO DO	9
☒ SCRUM - 11 Optimizer		TO DO	10
☒ SCRUM - 28 Data Visualizer tools		IN PROGRESS	8
☒ SCRUM - 10 Create a branch for the Result Data Manager		TO DO	5
☒ SCRUM - 26 Result Data Manager Design tools		IN PROGRESS	-
☒ SCRUM - 78 Create a layout of the report in LaTeX		TO DO	-
☒ SCRUM - 80 Clean up of the current main branch		TO DO	7
☒ SCRUM - 81 Create a copy branch named Version 1		TO DO	6
☒ SCRUM - 14 Implement Scenario #1		TO DO	8
☒ SCRUM - 79 Implement Scenario #2		TO DO	8
☒ SCRUM - 82 Merge the code		TO DO	-
☒ SCRUM - 83 DV - Create a branch for Data Visualizer		TO DO	5
☒ SCRUM - 84 Create a branch for Optimizer		TO DO	5
☒ SCRUM - 85 Create Generic Unit model		TO DO	7
☒ SCRUM - 86 Create Combustion Unit model		TO DO	7
☒ SCRUM - 87 Create HybridUnit Class		TO DO	7
☒ SCRUM - 88 Create ElectricUnit Class		TO DO	7
☒ SCRUM - 89 Create Generic to Optimized Unit adapter		TO DO	8
☒ SCRUM - 90 Create the ResultDataManager class in the "Services" folder		TO DO	8
☒ SCRUM - 91 Create the ResultDataSaver class in "Data" folder		TO DO	8
☒ SCRUM - 92 DV - Create DataVisualizer class		TO DO	7
☒ SCRUM - 93 DV - Setup charting library and replace placeholder in MainWindow.axaml		TO DO	8
☒ SCRUM - 94 DV - Implement SetAssets()		TO DO	6
☒ SCRUM - 95 DV - Implement SetSourceData()		TO DO	7
☒ SCRUM - 96 DV - Implement SetResultData()		TO DO	7
☒ SCRUM - 97 DV - Create ChartCommand object		TO DO	6
☒ SCRUM - 98 DV - Implement ShowChart(ChartCommand command)		TO DO	9
☒ SCRUM - 99 DV - Chart - Heat Demand		TO DO	6
☒ SCRUM - 100 DV - Chart - Electricity Prices		TO DO	6
☒ SCRUM - 101 DV - Chart - Net Heat Production per unit		TO DO	6
☒ SCRUM - 102 DV - Chart - Net Electricity Production/Consumption per unit		TO DO	6
☒ SCRUM - 103 DV - Chart - Expenses & Profit		TO DO	6
☒ SCRUM - 104 DV - Chart - CO2 Production per unit		TO DO	6
☒ SCRUM - 105 DV - Chart - Primary Energy / Fuel Consumption per unit		TO DO	6
☒ SCRUM - 106 DV - Chart - Net Production Costs per unit		TO DO	6
☒ SCRUM - 107 DV - Create DataVisualizer View with view switcher		TO DO	8
☒ SCRUM - 108 DV - Implement ShowHeatingGrid() and bind to image panel in UI		TO DO	6
+ Create			

Figure 3.5: Screenshot of the backlog after starting the 3rd sprint.

3.4 Sprint 4

The main goal of Sprint 4 was to finalise and test the product. The nature of the tasks aligned better with two teams of three group members instead of the previous division into three pairs, which matched the suggestion from the previous Sprint Retrospective:

- **Team Slovaks** – Daniel Falat, Dominik Bezo and Zoja Angelovicova
- **Team International** – Maria Tsioni, Pavel Seliavko and Zoe Elea Schiw

Daniel Falat took the Scrum Master role and Dominik Bezo took the role of the

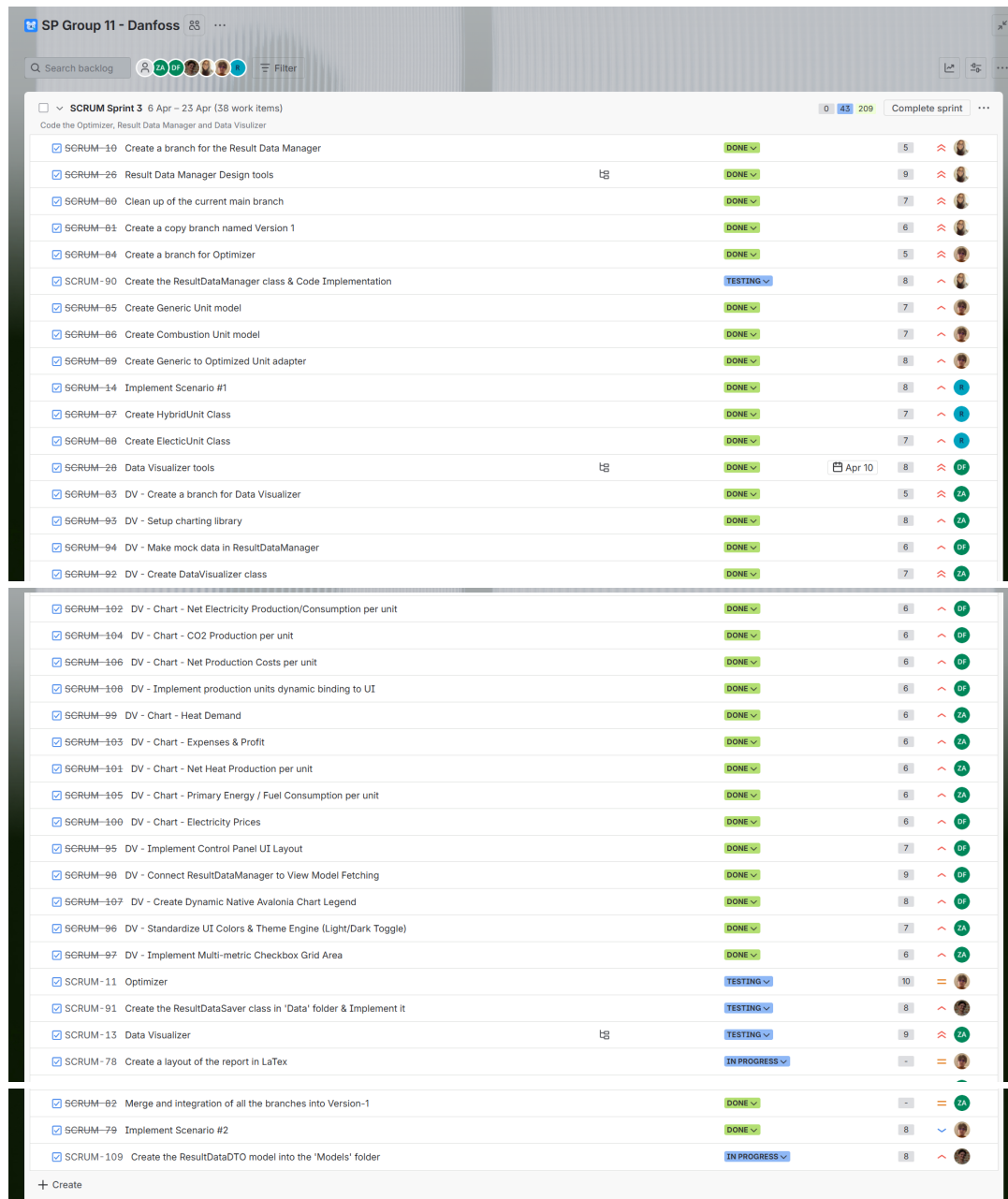


Figure 3.6: Screenshot of the backlog on the 16th of April.

Meeting Coordinator. Planned work included switching between Scenario 1 and Scenario 2, refactoring of the Optimizer, removing redundancy in the TimeSeries, implementing maintenance selection, updating the Result Data Manager headers, applying the UML feedback from the lecturer, and writing unit, functional and integration tests for every module. Time estimation and the time actually spent were also added to the tickets for the first time.

Because the focus was heavily directed at making the technical product fully working, the tickets regarding report-writing originally planned for this sprint had to be returned

Table 3.1: Start / Stop / Continue table for the 3rd Sprint Retrospective.

Start	Stop	Continue	Solutions
Implement ticket IDs	Having too long standup meetings; monitor with a timer so they do not exceed the 15 min limit	Pair programming (but also work individually)	Next sprint keep work in groups of 3
Update UML while coding, do not wait for next Sprint	Test the code before the end of the sprint so it is ready for the merge	If unable to test thoroughly, review	Documenting
More breaks during the meeting, e.g. 10 min break every hour	Leaving work for the last days	Ceremonies	
Stricter deadlines on finishing the PI		Keep the more detailed sprint planning phase	Make a schedule for the whole sprint in a meeting after sprint planning
Work planning before sprint planning for each individual team		Keep the good engagement and initiative	Make a schedule for the whole sprint in a meeting after sprint planning
Clear communication		Contact other people when in doubt	When receiving information, double-check to avoid confusion; do not make assumptions
Check what has already been agreed in meeting logs before acting	Assumptions in communication		
Use the team calendar			
Assign reviewers			
Suggestion: team of 3 people for the next sprint			Teams of 3 for unit tests in Sprint 4
Update each other when things happen, do not wait for the meetings			

to the backlog at the end of the sprint.

All work Apps Share

Basic JQL Search work Space Assignee Type Status Key More filters Clear filters Save filter

Work	Assignee	Priority	Status	Created	Due date	
☐ SCRUM-141 Validate merged code of both teams	Unassigned	Medium	DONE	Apr 23, 2026, 1:20 PM	Apr 29, 2026	...
☐ SCRUM-140 Merge TS's code into Version-1	Unassigned	Medium	DONE	Apr 23, 2026, 1:18 PM	Apr 29, 2026	...
☐ SCRUM-139 Merge TI's code into Version-1	Unassigned	Medium	DONE	Apr 23, 2026, 1:15 PM	Apr 29, 2026	...
☐ SCRUM-138 Create project with tests	Pavel Seliavko	High	DONE	Apr 22, 2026, 2:51 PM	Apr 26, 2026	...
☐ SCRUM-137 Create branch for implementing the maintena...	Dominik Bežo	Medium	DONE	Apr 22, 2026, 2:05 PM	None	...
☐ SCRUM-136 OPT - Integration Test - GetResultData() - ret...	Maria Tsioni	Medium	DONE	Apr 21, 2026, 10:08 PM	Apr 28, 2026	...
☐ SCRUM-135 OPT - Integration Test - GetResultData() - ret...	Pavel Seliavko	Medium	DONE	Apr 21, 2026, 10:08 PM	Apr 28, 2026	...
☐ SCRUM-131 Functional Test - GUI	Zoja Angelovicova	Medium	DONE	Apr 21, 2026, 9:48 PM	Apr 29, 2026	...
☐ SCRUM-130 Functional Test - SDM	Daniel Falat	Medium	DONE	Apr 21, 2026, 9:48 PM	Apr 29, 2026	...
☐ SCRUM-129 Functional Test - AM	Dominik Bežo	Medium	DONE	Apr 21, 2026, 9:48 PM	Apr 29, 2026	...
☐ SCRUM-126 Unit Test - GUI	Zoja Angelovicova	Medium	DONE	Apr 21, 2026, 9:47 PM	Apr 29, 2026	...
☐ SCRUM-127 Integration test - SDM	Daniel Falat	Medium	DONE	Apr 21, 2026, 9:47 PM	Apr 29, 2026	...
☐ SCRUM-126 Integration Test - AM	Dominik Bežo	Medium	DONE	Apr 21, 2026, 9:47 PM	Apr 29, 2026	...
☐ SCRUM-125 Update the UML diagram	Zoja Angelovicova	Medium	DONE	Apr 21, 2026, 9:46 PM	Apr 26, 2026	...

29 of 29

All work Apps Share

Basic JQL Search work Space Assignee Type Status Key More filters Clear filters Save filter

Work	Assignee	Priority	Status	Created	Due date	
☐ SCRUM-124 Update headers for the RDM	Dominik Bežo	Low	DONE	Apr 21, 2026, 9:45 PM	Apr 26, 2026	...
☐ SCRUM-123 Bind maintenance selection from UI	Daniel Falat	Highest	DONE	Apr 21, 2026, 9:45 PM	Apr 26, 2026	...
☐ SCRUM-122 Implement maintenance selection in OPT	Daniel Falat	Highest	DONE	Apr 21, 2026, 9:45 PM	Apr 26, 2026	...
☐ SCRUM-121 Merge MVVM Tests branch into General Tests...	Ria	Medium	DONE	Apr 21, 2026, 9:36 PM	Apr 29, 2026	...
☐ SCRUM-120 Relay Scenario Change from DV to AM	Maria Tsioni	Highest	DONE	Apr 21, 2026, 9:21 PM	Apr 24, 2026	...
☐ SCRUM-119 Create Scenario Request DTO for AM	Pavel Seliavko	Highest	DONE	Apr 21, 2026, 9:19 PM	Apr 24, 2026	...
☐ SCRUM-118 MVVM Functional Tests (positive & negative c...	Ria	Medium	DONE	Apr 21, 2026, 9:11 PM	Apr 29, 2026	...
☐ SCRUM-117 MVVM Unit Tests (positive & negative cases)	Maria Tsioni	Medium	DONE	Apr 21, 2026, 9:11 PM	Apr 29, 2026	...
☐ SCRUM-116 Create Integration Tests for OPT	Pavel Seliavko	Medium	DONE	Apr 21, 2026, 9:11 PM	Apr 29, 2026	...
☐ SCRUM-115 Create Unit Tests for OPT	Pavel Seliavko	Medium	DONE	Apr 21, 2026, 9:11 PM	Apr 29, 2026	...
☐ SCRUM-114 Create Integration Tests for RDM	Maria Tsioni	Medium	DONE	Apr 21, 2026, 9:11 PM	Apr 29, 2026	...
☐ SCRUM-113 RDM - Integration Test - GetResultData() - ret...	Maria Tsioni	Medium	DONE	Apr 21, 2026, 9:11 PM	Apr 28, 2026	...
☐ SCRUM-112 TimeSeries<T> remove redundancy	Ria	Low	DONE	Apr 21, 2026, 9:10 PM	Apr 29, 2026	...
☐ SCRUM-111 Optimizer check optimization correctness	Maria Tsioni	Highest	DONE	Apr 21, 2026, 9:10 PM	Apr 23, 2026	...

29 of 29

Figure 3.7: Completed work items from Sprint 4.

All work Apps Share

Basic JQL Search work Space Assignee Type Status Key More filters Clear filters Save filter

Work	Assignee	Priority	Status	Created	Due date	
☐ SCRUM-144 Report Writing - Zoe - [parts]	Ria	High	TO DO	Apr 26, 2026, 9:25 PM	May 03, 2026	...
☐ SCRUM-143 Report Writing - Pavel - [parts]	Pavel Seliavko	High	TO DO	Apr 26, 2026, 9:25 PM	May 03, 2026	...
☐ SCRUM-142 Report Writing - Maria - [parts]	Maria Tsioni	High	TO DO	Apr 26, 2026, 9:25 PM	May 03, 2026	...
☐ SCRUM-134 Report Writing - Zoja - [parts]	Zoja Angelovicova	High	TO DO	Apr 21, 2026, 9:56 PM	May 03, 2026	...
☐ SCRUM-133 Report Writing - Daniel - [parts]	Daniel Falat	High	TO DO	Apr 21, 2026, 9:56 PM	May 03, 2026	...
☐ SCRUM-132 Report Writing - Dominik - [parts]	Dominik Bežo	High	TO DO	Apr 21, 2026, 9:56 PM	May 03, 2026	...

6 of 6

Figure 3.8: Incompleted work items from Sprint 4.

The main conflict of the sprint surfaced during the Task Review: the two teams had

independently produced very similar solutions for different tasks (SCRUM-117 & 118 and SCRUM-128 & 131), creating duplicate code. The conflict was resolved through a discussion among the whole team where it was decided that, for documentation reasons, the best resolution was to keep the duplicate implementation while only keeping one solution active. By the Sprint Review on the 1st of May the Minimum Viable Product was complete and verified through testing. Report writing had to be transferred to Sprint 5. The team agreed that a heavyweight Start / Stop / Continue retrospective was unnecessary this sprint because collaboration inside both teams had worked well.

3.5 Sprint 5

Sprint 5 focused mainly on the division of the report tasks, report-writing and refactoring of the code. Because of the Math 2 exam on the 8th of May, a thorough Sprint Planning meeting was held before that on the 3rd of May. The sprint was formally started on the 9th of May, with the end date moved to the 17th of May. Maria became Scrum Master and Zoja the Meeting Coordinator, while Pavel remained the Product Owner. The two teams from Sprint 4 were kept.

On the code side, Team Slovaks took the task of making the Data Visualizer call the Optimizer only once with a clearer separation of concerns, and Team International took the refactoring of the Unit models to use more abstraction/inheritance and fewer interfaces. For the report, every chapter was assigned to an individual writer and reviewer (see Appendix), except for Chapter 5 (Implementation), which is co-written by both teams. A dedicated meeting was scheduled for the 16th of May to produce the final Class, Activity and Sequence diagrams used in the report.

During the Sprint Review and Retrospective, no major issues were identified, but the report-writing tickets had to be moved again to the next sprint, with the tickets already being in progress.

3.6 Sprint 6

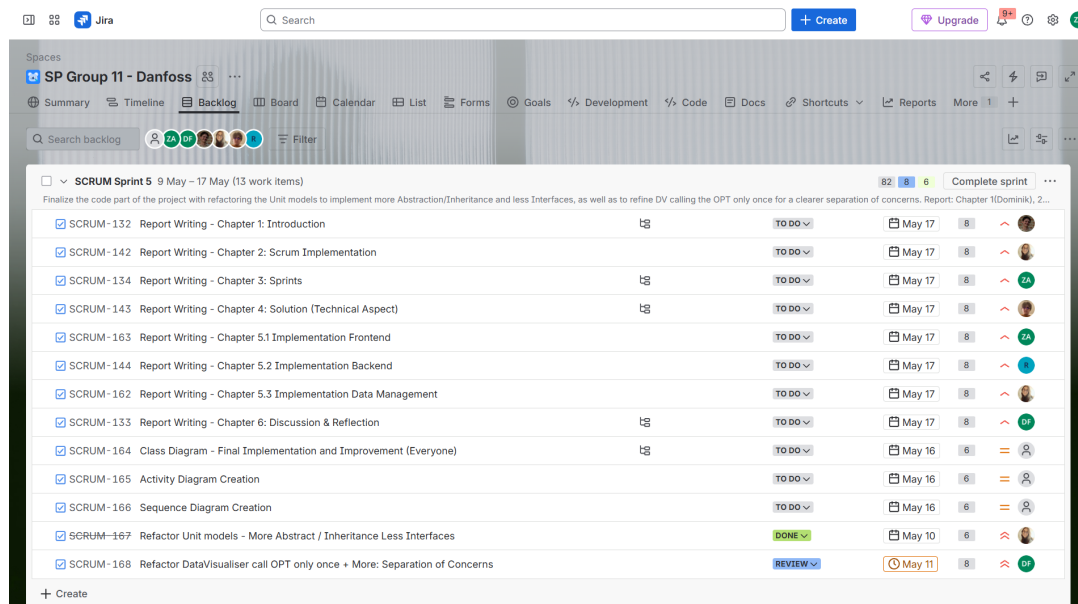


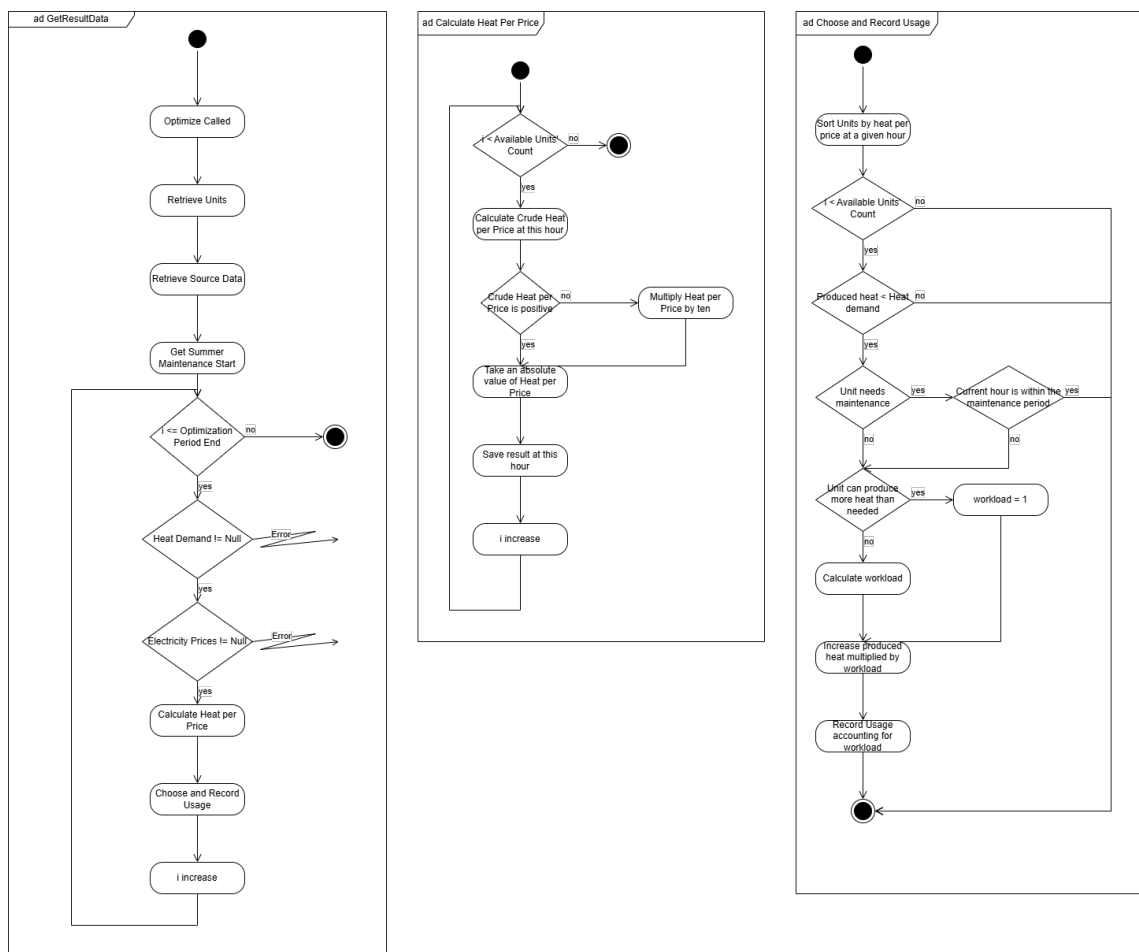
Figure 3.9: Screenshot of the backlog after starting the 5th sprint.

Chapter 4

Solution

4.1 UML Diagrams

4.1.1 Activity Diagrams:



These diagrams represent the optimization algorithm. The process happens inside the optimizer class, it starts when the `GetResultData` method of the optimizer is called. That calls the `Optimize` method, which proceeds to retrieve all data necessary for optimization to commence.

When data has been gathered it enters a loop, going through every hour of every day of the optimization period. In the loop it checks if the gathered data is workable, after, if data is correct, the process continues. The weighted ratio of heat to price of each unit is calculated, then the optimal unit is selected to be used in heat production and usage is recorded. This happens on a per hour basis by calling `CalculateHeatPerPrice` and `ChooseAndRecordUsage` methods.

`CalculateHeatPerPrice` method loops through the units, for each unit the ratio is calculated by first getting a crude heat divided by price, then the crude ratio is weighted by a factor of ten if it is negative. The weighting is performed to increase the unit's priority, because negative heat per price means that the unit is making money instead of costing money.

The `ChooseAndRecordUsage` method first sorts units by their heat per price ratio. When the list is sorted it enters a loop going through the sorted list, keeping track of heat demand, how much heat has been produced and how many units are available. Loop stops if it runs out of units or if the heat demand is reached. At the start of the loop it checks if the current unit is on maintenance, if unit is on maintenance it is skipped, otherwise loop proceeds. The next step is to check if the unit's maximum heat production exceeds remaining heat demand, which is the difference of total heat demand and produced heat. This is done to calculate workload - a coefficient from 0 to 1 that represents how much unit is used. If heat production is not exceeded the workload is 1, otherwise it is calculated by dividing remaining heat demand by the current unit's maximum heat production. After calculating workload, produced heat is increased by current unit's maximum heat production multiplied by workload, and then all unit's relevant characteristics, such as pollution, heat, fuel consumption, are recorded multiplied by workload.

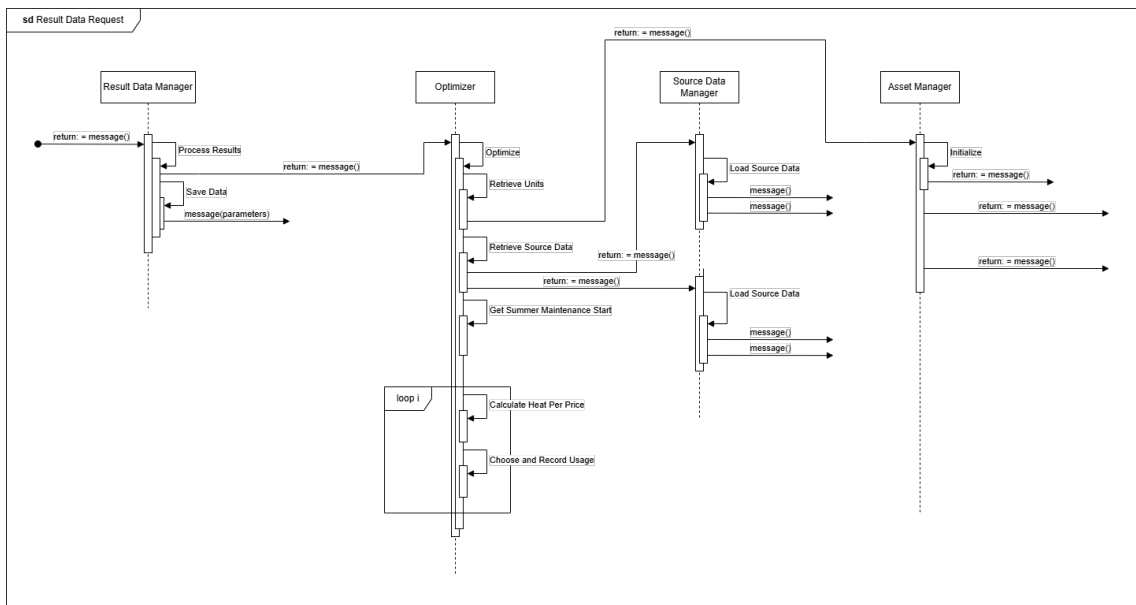
4.1.2 Sequence Diagram:

This sequence diagram represents how service classes interact when result data is requested.

Requests can come from several parts of the Presentation layer, which is represented as a found message.

The Result Data Manager makes a self-call, which then calls `Optimizer`. After that is resolved, within the same self-call another self-call is made to save Optimization results in local .csv files. For data management a call to a separate class is made, represented as a lost message. After that, the Result Data Manager's sequence ends.

The `Optimizer` makes a self-call that then makes three self-calls, first resulting in a



call to the Asset Manager and second in two calls to the Source Data Manager. It then enters a loop of two self-calls. After that Optimizer's sequence ends.

The Asset Manager makes a self-call to initialize. Initialization uses a separate class for data management, this is represented as a lost message. After that is resolved Asset Manager uses two separate classes to provide the correct scenario, and units of correct classes - those are represented as lost messages. After that Asset Manager's sequence ends.

Source Data Manager sequence starts when it receives the first call from Optimizer. This results in a self-call that uses a separate class for data manipulation represented as two lost messages. Then a second call from the optimizer results in following the same sequence. After that Source Data Manager's sequence ends.

4.1.3 Class Diagram:

This class diagram represents the core of the solution: services and models.

AssetManager has a one-way association with the Optimizer. This one-way association shows that AssetManager is called by the Optimizer, but doesn't call it. AssetManager aggregates instances of GenericUnit and Grid. This aggregation relationship shows that those are independent concepts.

SourceDataManager has a one-way association with the Optimizer. This shows that Optimizer can call SourceDataManager, but SourceDataManager doesn't call Optimizer. It aggregates the TimeSeries<T> class. This represents that TimeSeries<T> is an independent class.

Optimizer has one-way associations with AssetManager, SourceDataManager and

ResultDataManager. Those represent that Optimizer calls AssetManager and SourceDataManager, but is not called by them. And that shows that Optimizer is called by ResultDataManager, but doesn't call it. Optimizer aggregates TimeSeries<T> and OptimizedUnit. The aggregation relationship shows that both are independent concepts.

ResultDataManager has a one way association with the Optimizer. This shows that Optimizer is called by ResultDataManager, but ResultDataManager is not called by Optimizer. ResultDataManager aggregates OptimizedUnit. This aggregation relationship shows that this is an independent class.

Unit models form an inheritance tree. GenericUnit class and abstract OptimizedUnit classes inherit from the base abstract class Unit. GenericUnit represents units with characteristics including an image, while OptimizedUnit represents units with only performance specifications, and records of usage. From the abstract OptimizedUnit class three classes: CombustionUnit, HybridUnit and ElectricUnit, inherit to allow for the differences in how they calculate price and how they record usage.

4.2 Technical Architecture

Project was coded in C# in .NET framework, using Avalonia UI in combination with OxyPlot, ReactiveUI and CommunityToolkit.Mvvm for Data Visualization. All input and output data, specifically: images, units' specifications and resulting schedule are stored locally, file manipulation is performed by use of System library and CsvHelper library. Tests were implemented in the xUnit framework.

Solution is separated in following layers:

- Data Handling
 - Data
 - Assets
- Business Logic
 - Models
 - Services
 - ViewModels
- Presentation
 - Views

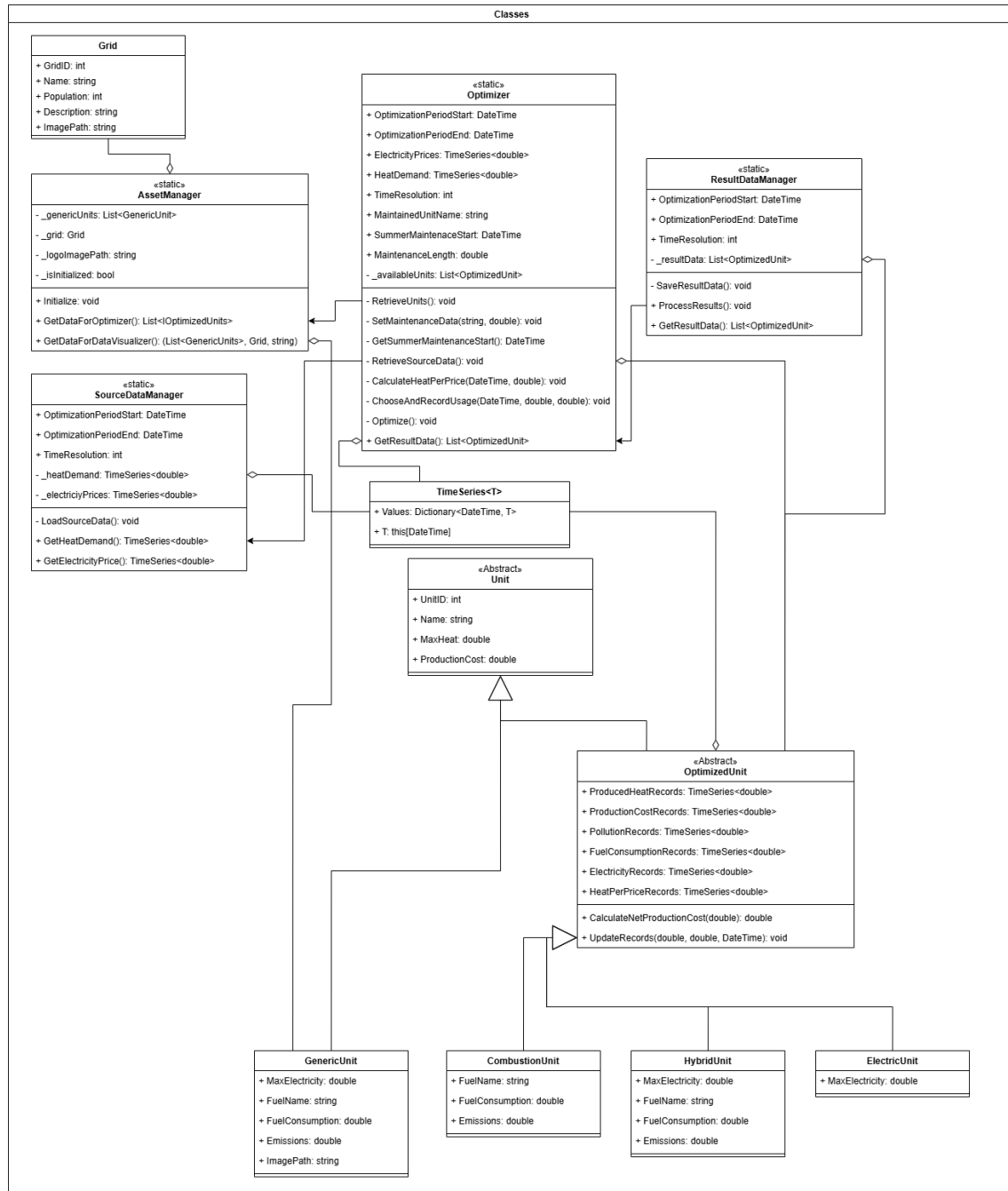
The project's architecture is a combination of the MVVM pattern and micro-service architecture approach.

Data Handling layer includes: Data - classes that handle file manipulation; Assets - static data storage consisting of .json, .csv and .png files.

Business Logic includes: Models - classes that: model real life objects and concepts, transfer data, represent UI components; Services - classes that manage classes from Data, and control the process of optimization; ViewModels - classes that provide functionality to the UI and establish connection between Models, Services and Presentation layer.

Presentation layer includes: Views - partial classes that control the appearance of the UI.

This structure was chosen because it follows SOLID principles which facilitate flexibility and scalability, by significantly reducing required amount of refactoring in development process. MVVM pattern was implemented in the overall structure of this project. Core benefit of MVVM is separation of responsibilities. Microservice architecture is most represented by Services and it follows the Open/Closed principle.



Chapter 5

Implementation and Technical Realization

5.1 Implementation

5.1.1 Implementation Frontend

Architecture and Technology Stack (MVVM in Avalonia UI)

The client application is built on the .NET platform using the Avalonia UI framework. Avalonia was chosen for its consistency with the Advanced Object-Oriented Programming course curriculum. The layout is described in XAML, so the structure of the interface is kept in markup files and the C# code-behind only contains what cannot be expressed declaratively.

The internal structure follows the Model–View–ViewModel (MVVM) pattern. MVVM was selected because the user interface and the optimization logic had to be developed in parallel by two sub-teams, and the pattern enforces a clear boundary between them. Keeping the View independent from the optimization code also made it possible to test the ViewModels in isolation, without instantiating any Avalonia controls. The three layers are organised as follows:

- **View:** The interface is defined in `MainWindow.axaml`. The file contains only XAML — layouts, styles, themes and bindings to the corresponding ViewModels. Its code-behind, `MainWindow.axaml.cs`, is intentionally kept minimal and only handles operations that cannot be expressed through bindings, such as loading the logo asset and switching between the application's pages.
- **ViewModel:** `MainWindowViewModel.cs` acts as the root ViewModel and owns the nested ViewModels used by the rest of the application. The two main ones are `DataVisualizerViewModel.cs`, which holds the state and configuration of the interactive charts, and `UnitViewModel.cs`, which exposes the parameters

and operational constraints of an individual heat production unit to the View.

- **Model:** The data layer is represented by the `DataVisualizerModel.cs` class, which contains time series data, data on previous operations, and data from the optimizer. This class provides access to the data needed for further visualization in the form convenient for use in the ViewModels.

Interaction between all three layers in the architecture is provided via the ReactiveUI library. The observable properties of both Models and ViewModels are created using the `RaiseAndSetIfChanged` method, which raises the property changed event each time the corresponding value changes. Thus, any change in the View, such as changing the slider to maintain a particular heat-producing unit, changes the value in the ViewModel that in turn recalculates the results of the Optimizer and the plot. Therefore, there is no need to explicitly call refreshing in the View.

5.1.2 Chapter name

5.1.3 Chapter name

5.1.4 Data Flow

During the architecture design phase, the main goal of the team was to clearly separate the presentation layer (frontend) from the complex mathematics of the optimisation algorithms (backend). This was achieved by implementing a strict data flow structure.

1. Initialisation

After the app is launched, `MainWindowViewModel` becomes the most important class, since it essentially controls the app flow. This ViewModel requests a list of all available production units using the Asset Manager's `AssetManager.GetDataForDataVisualizer()` method. In order for the app to load quickly and remain smooth, the `AssetManager` class is designed to cache all loaded data. When the app is launched for the first time, it uses the `AssetsLoader` class to read and parse the data from JSON files. Whenever the `AssetManager` is called again, it does not need to read the data from disk, but rather returns the data directly from the cache. This approach helps to decrease latency as much as possible and ensures that the UI loads more quickly, even with a growing number of production units.

2. Optimisation

The optimisation data cycle begins whenever the operator uses the control panel to change the configuration. It can be initiated by either changing the scenario, switching the optimisation period between summer and winter, or adjusting the maintenance settings.

- `MainWindowViewModel` detects changes in the configuration and launches

the `RefreshDataAndChart()` method. This method handles synchronisation of the scenario configuration using the `ScenarioManager.CurrentScenario` method. Afterwards, the `SyncOptimizationPeriod()` method is called, which defines clear time boundaries for both the winter and summer periods. The summer period ranges from 8 September to 22 September, while the winter period ranges from 5 January to 19 January. Maintenance settings are sent to the Optimizer module using the `Optimizer.SetMaintenanceData()` method.

- After all configuration data is saved, `MainWindowViewModel` instructs `DataVisualizerModel` to update the data using `Visualizer.Model.UpdateData()`. At this stage, `DataVisualizerModel` requests data from `ResultDataManager`, which starts the optimisation process. The goal of the optimisation is to meet the heat demand using the available production units. The `Optimizer` calculates workload and then selects the most profitable configuration of units to activate.
- When the optimisation process is complete, `ResultDataManager` saves the result data into CSV files using the `SaveResultData()` method. These records serve as a debugging tool if the optimisation does not run correctly or for occasional audits.
- In the end, the `DataVisualizerModel` receives the result data in form of `OptimizedUnit`. It prepares the data for the `DataVisualizerViewModel`, where the resulting `TimeSeries` data are converted into curves and bars on the chart. The `UpdatePlot()` method links units with the correct colours from `ChartColors.cs`, sets up axes using `Axes.cs`, the legend using `Legend.cs`, and calls `InvalidatePlot(true)` to ensure that the updated graph is displayed immediately within the GUI.

Conclusion of the Data Flow

Due to this strictly engineered data architecture, the entire 14-day optimisation, including exporting data into CSV files and rendering of complex graph usually takes less than 100 milliseconds. The operator therefore receives immediate feedback which ensures a fluid and positive user experience.

5.1.2 Implementation Backend

The core algorithm of the application is the Optimization-process, which calculates, based on given Data from the Asset Manager and the Source Data Manager, the most optimal time schedule for the production units, with focus on the most cost-efficient outcome. Those calculations are affected by the choice of Scenarios and the maintenance of the production unit.

Data from Asset Manager

For the calculation of the time schedule, first the Scenario Manager provides a list of Generic units for the Asset Manager. The Asset Manager passes the list to the GenericToOptimizedAdapter, which identifies each unit in the given list as one of three types of units: HybridUnit, ElectricUnit or CombustionUnit. And creates a new list with the identified units. This list gets passed back to the Asset Manager, which provides it for the Optimizer request.

Data from Source Data Manager

Next to the list of units the Optimizer also depends on the Data of the heat demand, and for the second scenario the electricity prices, as well as the current period, summer or winter. Those Data get provided by the Source Data Manager, which first loads Data from a given from the Source Data Loader in form of Time series objects. These Objects contain a Date and time, the period and the value for the heat demand or the value for the electricity price and can be indexed by the DateTime value. The Source Data Manager contains two functions, one to load the electricity price and one to load the heat demand. Those functions get called by the Optimizer to obtain the required Data for the calculation.

Calculation for Optimization

For the optimization process the Optimizer contains a function that first calculates the value of heat per price for each available unit and adds it to the unit's properties as HeatPerPriceRecords. With these values the next function sorts through the list of available units and sorts for the HeatPerPriceRecords value, from most heat per cost to least money. It also calculates if the Unit can produce the required heat demand on its own or if additional units are required, by comparing the heat demand to the maximum of heat the unit can produce.

Optimization function

In the optimize function itself, the application calculates heat per price and sorts the list of available units for each hour in the selected duration. The optimization function gets called when the Result Data Manager requests the optimized data.

ResultDataManager

After calling the optimized data from the Optimizer, the Result Data Manager adds the number of used hours for each unit and saves the Data as CSV file. Those Data can be requested by the Data Visualizer.

Maintenance

The Optimizer Class contains a function that sets a selected unit on maintenance for a chosen number of hours. The start of the maintenance will be selected randomly by the program and is only possible during summer period due to less heat demand. During the sorting process, of the available units the function checks if each unit is on maintenance, if its true the loop continues with the next unit of the list and doesn't add the maintenance unit to the optimized list.

5.1.3 Implementation Data Management

5.1. IMPLEMENTATION AND TECHNICAL REALIZATION

Chapter 6

Discussion and Reflection

6.1 Scrum

During the development of the project, a Scrum agile was implemented to deliver structured and transparent software development. Over the semester, six sprints were completed. Each sprint contributed to the team's development, working methods, and the quality of the final semester project.

Reflection on Scrum practices and their benefits

Transparency and responsibility were ensured by rotating the role of Scrum Master each sprint (Dominik in S1, Pavel in S2, Zoja in S3, Daniel in S4, Maria in S5, Zoe in S6). This allowed each team member to take responsibility for all processes and sprint coordination. Using Jira provided a clear overview of task status, with each ticket having a clearly defined assignee or team and a reviewer to ensure quality.

In the first sprint, the workload distribution was underestimated. As a solution, a story points system was introduced, using a scale of 1–10 based on the sum of difficulty (1–5) and importance (1–5). This system greatly improved our planning and monitoring of burn-down charts. Later, also deadlines for tasks were used to help review them on the go, so that there always was enough time to fix issues the development team or pair might have missed.

Throughout the development phase, team and pair programming were utilized. The first three sprints were pair-based, with pairs changed twice. In sprints 4–6, workload was divided between 2 teams of three (team Slovaks and team Internationals). This technique proved effective and helped us eliminate technical problems from the early sprints, especially those that were related to time management.

Observed changes in behaviour and code quality

By implementing code reviews and coding principles in the second sprint, a higher consistency was reached, also a cleaner and easier-to-read code was produced to help understand everything for those who had not written that part.

After facing difficulties in the initial phase such as missing feedback from each other during meetings and passivity, the team implemented a rule to use webcams. This solution also proved to be effective and improved the sense of presence and teamwork.

Difficulties and issues

During the development, the team faced several issues, for example, in Sprint 2, tickets needed to be added in the middle of the process because the technical difficulty was not estimated correctly. Team's response was implementing more detailed sprint planning, dividing large and important tasks into smaller tickets that were more predictable.

Duplicity and assumptions were also problems that were faced during the development. In the fourth sprint, miscommunication and assumptions led the team to produce the same part of the code twice. This experience was very beneficial for learning and implemented stricter coordination between the Slovak and International teams in the later phase of project development.

During development, the team experienced issues with Jira, where tickets were sometimes not marked as done even after being moved to the "Done" column. This was later fixed, but the charts still did not reflect reality. There was another issue – the presence of subtasks within each ticket. It was addressed by transitioning to separate tickets in order to have the chart displayed correctly. Despite these solutions, the chart remained problematic although it was more realistic than in the beginning.

6.2 Requirements Engineering

The engineering process in the project was important in transforming the task into functional and professional software. This process helped maintain the correct scope of the user requirements and the technical implementation of the solution.

Impact of adopted practices on work quality

In the beginning, a noun-verb analysis was conducted. This technique helped in the early development stage by identifying important entities and their desired behaviour. As a result, the key responsibilities of each module (for example, Asset Manager, Source Data Manager) were precisely defined, which led to a clearer architecture and separation of concerns.

Creating the product backlog based on user stories helped with dividing the complex system into smaller, more testable functions. This ensured that during the development we remained focused on our goal: optimising heating expenses.

Before starting application development, the UI was prototyped. Each team member sketched their vision of the UI, after which the best design was picked and then improved by incorporating better ideas from other sketches. After creating final sketch, a proper Figma prototype for the GUI was created, which served as source of truth for UI. This process eliminated uncertainties in front-end development with Avalonia and saved time otherwise spent discussing design.

A decision to use UML diagrams and CRC cards as dynamic documents that evolved with the code development enabled the team to react more flexibly to new discoveries during implementation.

Important takeaways for future projects

In the future, it would be beneficial to keep the practice of reconsidering requirements in the beginning of each sprint. For example, in the second sprint it was discovered that in order to solve Data Visualizer module, a Source Data Manager had to be developed prior, which helped prevent blocking the progress of other teams.

Another takeaway is collaboration with Product Owner. During development, it proved to be necessary for clearing up uncertainties in some tickets. In the future team would plan to use this role even further and more intensively in agreeing acceptance criteria in certain tasks and in avoiding assumptions.

6.3 Refactoring

Refactoring in our project was one of the key processes, which continued in parallel with the development of new application features. Its goal was not only to clean up the code, but also to keep the code more maintainable and readable when different teams worked on the same parts of the project.

Effects of refactoring on our code quality

As mentioned in 6.1, a set of strict coding principles was developed, which were later applied in each refactoring. These principles mainly included reducing nesting to a maximum of 3 layers and extraction of duplicate code into separate methods.

The most important phase of refactoring occurred in Sprints 4 and 5, where after merging branches, the team had to clear up redundancies in TimeSeries which enabled easier data access and processing across different modules. We also refactored DataVisualizerViewModel for better maintainability and made changes to interfaces which were removed and the code was simplified with the same functionality.

Skills balance and teamwork

Refactoring was used as a tool to balance technical skills across group. Through the pair programming, less experienced members were able to learn best practices and coding standards from their peers while working on code clean-up.

Rotating work between teams, for example having Team Slovaks refactor modules initially developed by Team International, ensured that everyone understood overall architecture rather than just single module.

Furthermore, establishing the English as standard for all comments and naming (after listening to feedback from each other based on previous semester project) during our refactoring sessions significantly improved collaboration and understanding within an international team.

Challenges during refactoring

One of biggest challenges was keeping our documentation in sync with the code. As project evolved, UML diagrams and CRC cards had to be constantly updated to reflect latest refactored state, which was demanding to manage alongside development.

Another significant hurdle that was encountered was that the code was written in quite complicated way, even though it followed the coding principles that were agreed on. This complexity meant that the refactoring took much more time than initially expected. The team had to spend the extra effort to simplify the logic and make it more readable without breaking any of functionality we already implemented.

6.4 Code review

Code review was a necessary step before merging changes into active development branches (for example, “Version-1” and “Version-2” branches) and to confirm code quality. This process served not only for discovering mistakes, but also as a very important tool for peer learning.

Good effects for code quality and knowledge sharing

In the first sprint, a review system was introduced, where teams checked each other’s work (for example team Daniel & Dominik reviewed Zoe’s & Pavel’s work). This approach was later refined since we transitioned to 2 teams consisting of 3 members – Team Slovaks and Team Internationals. This approach improved collaboration and technical understanding within the team and ensured that code would be understandable even for someone who did not write certain parts of code directly.

During revisions, it was checked whether the code follows the guidelines set up in the second sprint, such as using guard clauses, avoiding nesting and correctly naming

variables and methods.

Code reviews allowed all members to understand functionality of individual managers (Asset Manager, Optimizer etc.), which facilitated later integration in Sprint 3 and 4.

Challenges and our solutions

In Sprint 3, the team realized that we forgot to assign reviewers to some tickets, which led to less transparent history of changes. In the following sprint, we corrected this by strictly assigning reviewers directly in Jira.

The review process in Sprint 4 helped with identifying a loss of productivity, when two teams developed similar solutions due to misunderstanding and assumptions that occurred during planning. Review helped to consolidate these solutions in time and proceed with the better solution.

Initial challenge was that commits in GitHub did not always include ticket ID from Jira. During retrospective a rule was introduced to use ticket code (in format of SCRUM-XX) in every commit name, which significantly helped with tracking of changes during reviews.

Overcoming obstacles

Instead of authoritarian approach, the team conducted constructive dialogue during reviews. Reviews were almost always done with presence of both teams and if problem occurred, it was solved immediately, therefore, members did not have to wait for final sprint meeting and could continue working on other tickets and use time more effectively.

Another habit that team took on was ensuring code quality. During review, if the produced code did not meet team's standards, tickets remained in Review category and were not moved to Done until final approval by whole team or even put back to backlog and worked on in the future sprints.

6.5 Testing

Testing was an important part of the project development to make sure the heat distribution optimization system works as expected. We decided to go with a testing strategy that included unit, integration, and functional tests.

Before diving into the details, it has to be mentioned that the team faced quite a challenge with unit testing. Since a large part of the code was written using static and private classes and methods, it was very difficult to isolate individual components for pure unit tests. Because of this static architecture and how specific the logic was, the team realized that traditional unit testing would not be suitable for our architecture. Instead of forcing tests that didn't fit the current structure, after consultation with

the professor, it was decided to focus our energy on integration and functional tests. This way, it could better be ensured that the different modules worked correctly together.

Test implementation

All the tests were built on the xUnit framework. For testing the GUI, the `Avalonia.Headless.XUnit` extension was used, which allows testing the project without the need to physically open the application window.

As mentioned before, we agreed on using a strict naming format for tests: `Object_Method_ExpectedOutcome`. A good example of the test naming format can be `ToggleLegend_FlipsIsLegendExpanded_Correctly`. This simple rule made all of the test reports much easier to read and understand.

Because some tests interacted with static resources and shared states (like `Optimizer` or `AssetManager`), the “[Collection(“Sequential”)]” attribute had to be used. This prevented conflicts and crashes during parallel execution.

The scope of tests

For unit tests, those were focused on the logic inside `ViewModels` and auxiliary services. A good example are tests for `DataVisualizerViewModel`, where it was tested whether theme colours switch correctly or if data aggregates correctly for summer and winter periods.

Integration tests checked whether all managers interacted with external files correctly. The team verified if `AssetManager` and `SourceDataManager` correctly load data from both JSON and CSV files and if `ResultDataManager` produces results that are not empty, right after optimization runs.

Functional tests were there to check whole business processes. For instance, tests confirmed that when selecting a Heat scenario, units like Gas Motor 1 or Electric Boiler 1 are correctly excluded from the calculation.

Pros and cons of testing

Thanks to the implemented tests, bugs were found much faster during refactoring, especially in complex math inside optimizer and when processing time series in `SourceDataManager`.

However, testing GUI was quite hard and intellectually demanding because it had to simulate how user navigates between pages using `CanGoNext` and `CanGoPrev`. Development teams also lost some time and productivity in Sprint 4 having to figure out what to do with duplicate tests caused by misunderstanding and assumptions from team Slovaks.

In future projects, it would be nice to try mocking (for example with Moq library) to isolate tested logic from file system, which would make tests even faster and more stable.

6.6 Future Work

In this final section of chapter 6, we reflect on overall outcome of the project compared to initial requirements and identify areas that have potential for future development and improvements of the system.

Fulfilment of requirements and problem solving

Our project successfully implemented all key managers (Asset, Source Data, Result Data) and functional algorithm in Optimizer. Product solves the main problem, which is defining heat schedules for all production units with lowest possible expenses and highest profit on electricity market. Switching between Scenario 1 (Heat) and Scenario 2 (Heat and Electricity) were fully implemented and tested. This allows system to dynamically respond to availability of technologies like electric boilers and gas motors. The resulting Data Visualization module presents time series representations of heat demand, production values, expenses, profit and CO2 emissions.

Identified problems and limitations

As mentioned in testing section, current architecture relies heavily on static methods and classes in managers. This led to need for sequential test execution to prevent conflicts. In future development, it might be better to transition to Dependency Injection architecture. Currently, Asset Manager and Source Data Manager rely on holding data in local files. This manual configuration with file paths makes it difficult to deploy application in different environment.

Suggestions for future development

To fix our issue with file paths, future version should use database to save data instead of files.

- Since current system works with historical 14-days winter and summer periods from Excel/CSV files, an API client could be implemented. This client would call API from Energinet.dk to directly read electricity prices from their web server. Furthermore, it will be possible to implement APIs for communication between the modules instead of calling methods directly.
- Another feature that was thought of is ability to activate (allow) and deactivate (prohibit) units, change their statistics and add create new units to be part of optimization directly from UI. This would make it much easier to simulate real

maintenance periods. It could then directly compare results in terms of revenue, CO₂ emissions and other metrics directly in our data visualization.

- Other than that, it is definitely possible to add map of the city into the application, with heating units and district heating network visually displayed as well as weather forecast by using meteorological API. Since heat demand usually depends on the temperature outside, using live weather data would make optimization much more realistic.
- Exporting is another step for future development. Adding an option to generate and export graphs and optimization results in various formats would be useful for company management and for presentations.
- Another feature that was thought of was user authentication and roles. Since application would be used by different employees, having basic login system with roles (like Admin who can edit production units and Viewer who can only see charts) would help with improving the security of the optimizer application.

Chapter 7

Conclusion

Appendix A

Appendix A

A.1 First section

A.2 Second section

A.3 Third section